

False Positive Documentation

Kindly provide the answers for the following queries regarding the vulnerability.

1. Name of the Application

Cotiza CPQ

2. Package ID/Version ID/Listing ID

04tao000005dAFxAAM

3. Partner Name

Cotiza

4. Document Filing Date (YYYY-MM-DD)

2026-07-06

5. Target URL (Optional)

6. Review Date (In Case if app already undergone security review)

2026-07-06

Repeat Step 7 to 9 in case of multiple Vulnerability

7. Vulnerability Name

pmd:ProtectSensitiveData

[Meant for the Internal Salesforce Security Team](#)

Final Comment from Salesforce Team	
Date of Review (YYYY-MM-DD)	
Result	

8. Detected By

- Chimera
- **PMD**
- Security Review
- Others,

If case of others Please provide details :

9. Detailed explanation for considering False Positive (Elaborate as much as possible)

False positive - System_Settings__c.Line_Item_Pricing_Fields_Refresh__c is a multiselect config field; Refresh substring is unrelated to auth tokens

10. Evidence (screenshots of the code, implementation, Command Output)

Line_Item_Pricing_Fields_Refresh__c.field-meta.xml X

Cotiza CPQ > main > default > objects > System_Settings__c > fields > Line_Item_Pricing_Fields_Refresh__c.field-meta.xml > CustomField

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <CustomField xmlns="http://soap.sforce.com/2006/04/metadata">
3   <fullName>Line_Item_Pricing_Fields_Refresh__c</fullName>
4   <description></description>
5   <externalId>false</externalId>
6   <inlineHelpText>When to run refresh pricing fields on QuoteLineItem based on the current values of the related PricebookEntry
7   <label>Line Item Pricing Fields Refresh</label>
8   <required>false</required>
9   <trackFeedHistory>false</trackFeedHistory>
10  <trackHistory>false</trackHistory>
11  <type>MultiselectPicklist</type>
12  <valueSet>
13    <restricted>true</restricted>
14    <valueSetDefinition>
15      <sorted>false</sorted>
16      <value>
17        <fullName>Quote Edit</fullName>
18        <default>false</default>
19        <label>Quote Edit</label>
20      </value>
21      <value>
22        <fullName>Contract Adjustment</fullName>
23        <default>false</default>
24        <label>Contract Adjustment</label>
25      </value>
26    </valueSetDefinition>
27  </valueSet>
28  <visibleLines>3</visibleLines>
29  <defaultValue></defaultValue>
30 </CustomField>
```

Repeat Step 7 to 9 in case of multiple Vulnerability

1. Vulnerability Name

pmd:VfUnescapeEI and Stored XSS — Section_Component.component /
Proposal_PDF_Word.page

[Meant for the Internal Salesforce Security Team](#)

Final Comment from Salesforce Team	
Date of Review (YYYY-MM-DD)	
Result	

2. Detected By

- Chimera
- **PMD**
- Security Review
- Others,

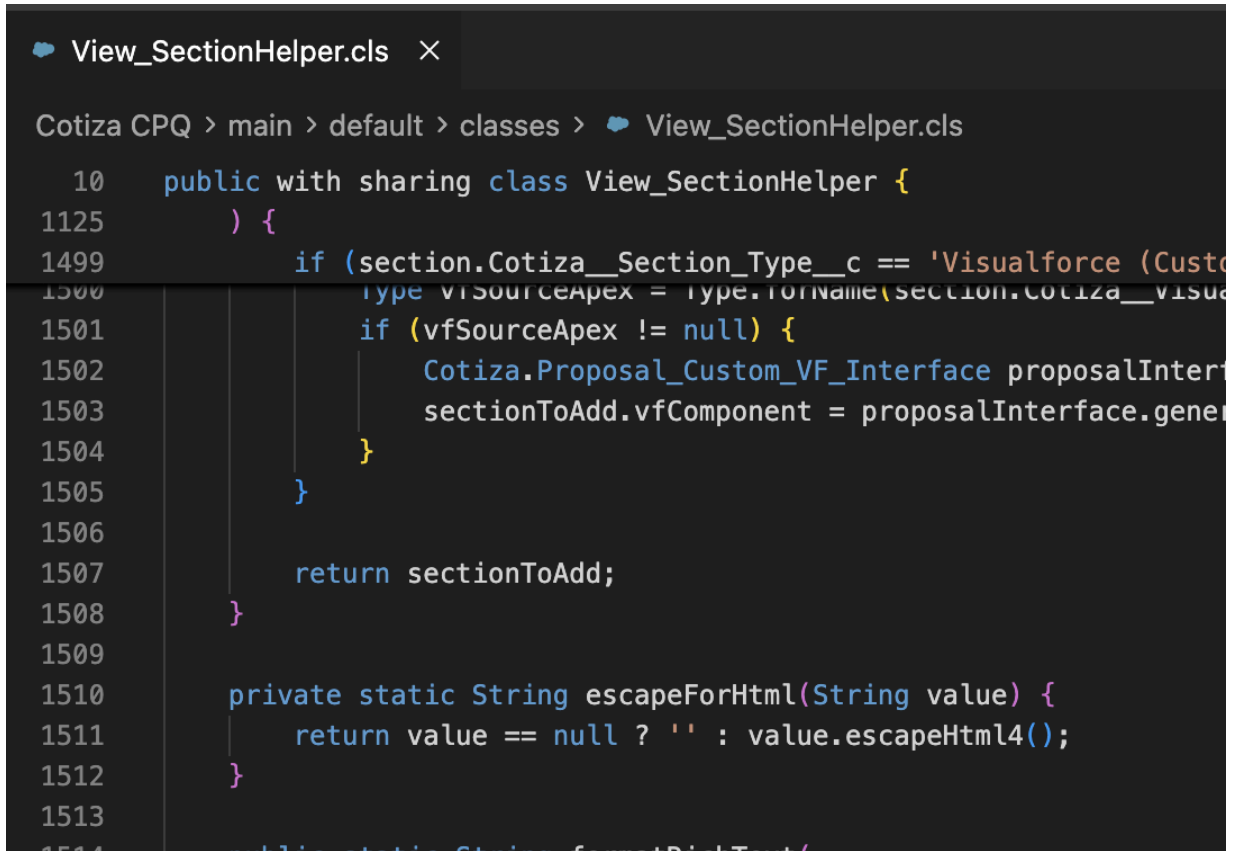
If case of others Please provide details :

3. Detailed explanation for considering False Positive (Elaborate as much as possible)

Fields are Salesforce Html-type playbook template content configured by authorized administrators. Dynamic piped values (quote answers, approval text) are HTML-encoded in View_SectionHelper.formatRichText (via View_SectionHelper.escapeForHtml) before insertion. Unescaped rendering is required to preserve admin-authored HTML formatting in

PDF/Word output. Output is generated server-side within an authenticated Salesforce session.

4. Evidence (screenshots of the code, implementation, Command Output)



```
View_SectionHelper.cls X
Cotiza CPQ > main > default > classes > View_SectionHelper.cls
10    public with sharing class View_SectionHelper {
1125    } {
1499        if (section.Cotiza__Section_Type__c == 'Visualforce (Custom
1500            type vfSourceApex = Type.forName(section.Cotiza__visual
1501            if (vfSourceApex != null) {
1502                Cotiza.Proposal_Custom_VF_Interface proposalInterf
1503                sectionToAdd.vfComponent = proposalInterface.gener
1504            }
1505        }
1506
1507        return sectionToAdd;
1508    }
1509
1510    private static String escapeForHtml(String value) {
1511        return value == null ? '' : value.escapeHtml4();
1512    }
1513
1514    private static String formatRichText(
```

```

View_SectionHelper.cls X
Cotiza CPQ > main > default > classes > View_SectionHelper.cls

10  public with sharing class View_SectionHelper {
1522 } {
1524     if (richText != null) {
1552         for (String questionVar : answerMap.keySet()) {
1555             if (answer != null) {
1576                 } {
1577                     richText = richText.replace('%'+questionVar+'%', escapeForHtml(answer.Cotiza__Value_Text__c));
1578                 }
1579             }
1580         }
1581
1582         // Pipe approval text if applicable
1583         if (approvalInfo != null) {
1584             for (String approvalPipe : approvalInfo.keySet()) {
1585                 String approvalValue = approvalInfo.get(approvalPipe) != null ? approvalInfo.get(approvalPipe) : '';
1586                 if (approvalPipe != 'opportunity' && approvalPipe != 'account') {
1587                     approvalValue = escapeForHtml(approvalValue);
1588                 }
1589                 richText = richText.replace('%Approval.' + approvalPipe + '%', approvalValue);
1590             }
1591         }
1592     }

```

```

View_SectionHelper.cls X
Cotiza CPQ > main > default > classes > View_SectionHelper.cls

10  public with sharing class View_SectionHelper {
1522 } {
1524     if (richText != null) {
1527         if (systemSettings.Cotiza__Proposal_Quote_Piped_Fields__c != null) {
1528             for (String quoteField : systemSettings.Cotiza__Proposal_Quote_Piped_Fields__c.split(';')) {
1544                 else if (fieldType == 'Text') {
1545                     richText = richText.replace('%Quote.' + quoteField + '%', escapeForHtml((String) quoteFieldValue));
1546                 }
1547             }
1548         }
1549     }
1550 }

```

Repeat Step 7 to 9 in case of multiple Vulnerability

1. Vulnerability Name

Apex CSRF Aura LWC — CRUDHelper.cls via LWC callers

[Meant for the Internal Salesforce Security Team](#)

Final Comment from Salesforce Team	
Date of Review (YYYY-MM-DD)	
Result	

2. Detected By

Chimera
PMD
Security Review
Others,

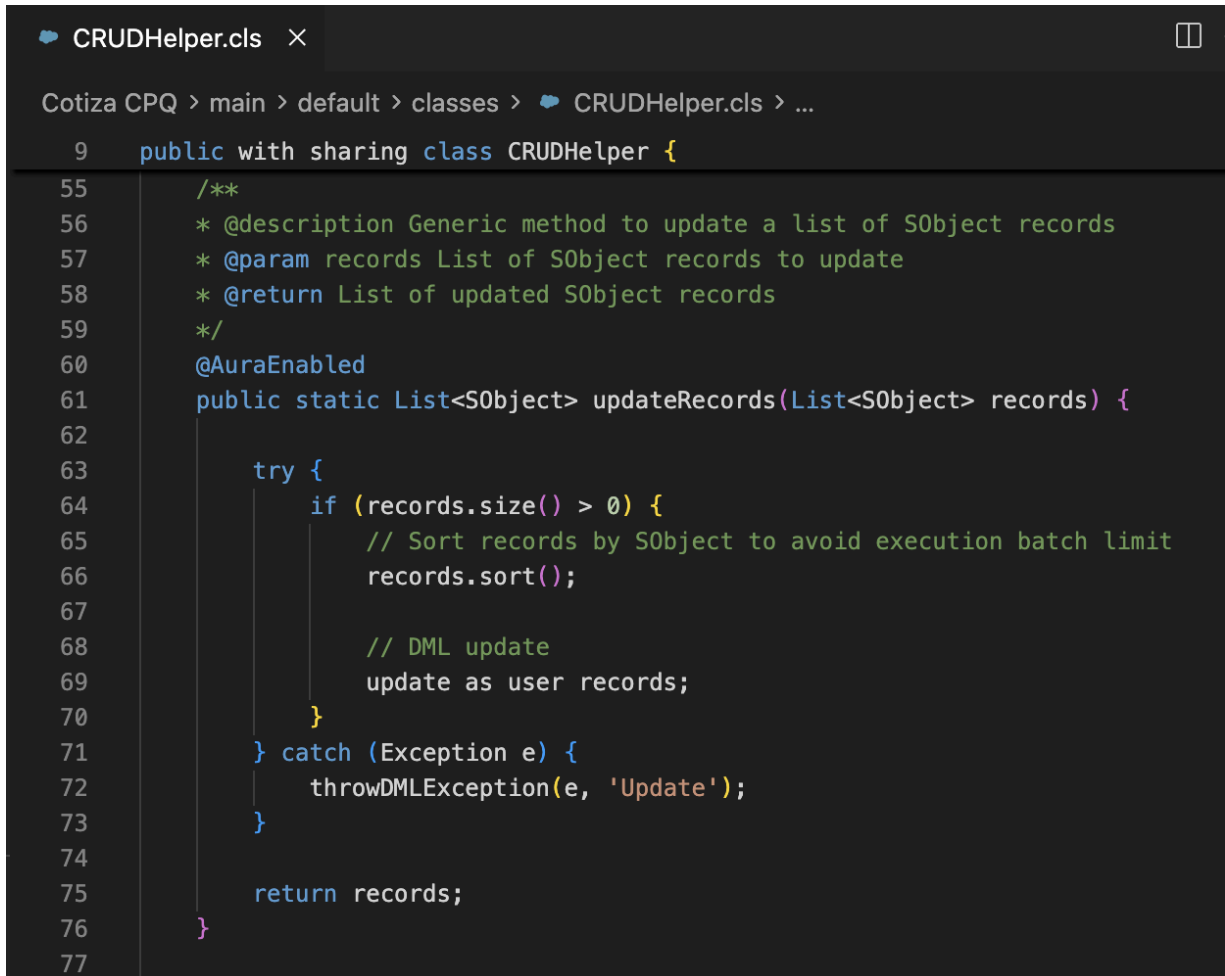
If case of others Please provide details :

3. Detailed explanation for considering False Positive (Elaborate as much as possible)

Checkmarx flags any @AuraEnabled Apex method performing DML when referenced from LWC, without analyzing whether the call is user-initiated. All flagged call sites are explicit user actions (void contract, approve/reject, save notes, sync/unsync quote, delete with confirmation modal, close preview). DML is never executed on component initialization. Apex runs with sharing using USER_MODE DML (as user), enforcing the caller's CRUD/FLS

permissions. Calls require an authenticated Salesforce Lightning session with platform-level CSRF/session validation. Components are deployed to internal Lightning record/app pages, not unauthenticated guest contexts.

4. Evidence (screenshots of the code, implementation, Command Output)



```
9 public with sharing class CRUDHelper {
55     /**
56     * @description Generic method to update a list of SObject records
57     * @param records List of SObject records to update
58     * @return List of updated SObject records
59     */
60     @AuraEnabled
61     public static List<SObject> updateRecords(List<SObject> records) {
62
63         try {
64             if (records.size() > 0) {
65                 // Sort records by SObject to avoid execution batch limit
66                 records.sort();
67
68                 // DML update
69                 update as user records;
70             }
71         } catch (Exception e) {
72             throw DMLEException(e, 'Update');
73         }
74
75         return records;
76     }
77 }
```

CRUDHelper.cls ×



Cotiza CPQ > main > default > classes > CRUDHelper.cls > ...

```
9 public with sharing class CRUDHelper {  
25 /**  
26  * @description Generic method to delete a list of SObject records  
27  * @param records List of SObject records to delete  
28  */  
29 @AuraEnabled  
30 public static void deleteRecords(List<SObject> records) {  
31  
32     try {  
33         if (records.size() > 0) {  
34             // Sort records by SObject to avoid execution batch limit  
35             records.sort();  
36  
37             // DML delete  
38             delete as user records;  
39         }  
40     } catch (Exception e) {  
41         throwDMLException(e, 'Delete');  
42     }  
43 }  
44 }
```